과제 운영 장애 주입 테스트 (실습)

> 제공된 장애 주입 코드로 운영 장애를 재현하고 trace로 실패 지점을 찾아 retry와 fallback으로 복구해봅니다.

실제 운영에서 도구는 반드시 실패합니다. 외부 API가 느려지거나(timeout) 호출 한도에 걸리고(rate_limit) 계약과 다른 형식의 응답이 오기도 합니다(bad_schema).

이 미션은 제공된 코드로 장애를 일부러 주입해 재현하고 복구를 확인합니다. 목표는 장애를 만드는 것이 아닙니다. **로그와 trace로 실패 지점을 찾고 retry(재시도)와 fallback(대신 쓸 도구로 전환)을 적용하는 능력**을 기르는 것입니다.

장애 종류마다 알맞은 복구 전략이 다릅니다.

장애	성질	복구 전략
timeout	일시적일 수 있음	제한 시간 설정 + 재시도(retry)
rate_limit	기다리면 풀림	대기 시간을 늘려 가는 재시도(exponential backoff)
bad_schema	기다려도 안 풀림	재시도 무의미. 응답 검증 후 fallback 도구로 전환

따라하기

1. 제공된 장애 주입 코드를 `day4_practice/failure_injection.py` 로 저장합니다. 환경 변수 FAILURE_MODE로 장애 종류를 골라 주입하는 구조입니다.

```
# failure_injection.py — 운영 장애 주입 도구 (제공 코드)
"""FAILURE_MODE 환경 변수로 장애 종류를 골라 주입합니다.
값: timeout | rate_limit | bad_schema | none

실행 예:
  $env:FAILURE_MODE="timeout"; python failure_agent.py
"""
import os
import time
import logging
from langchain_core.tools import tool

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
    handlers=[logging.FileHandler("agent.log", encoding="utf-8"), logging.St
reamHandler()],
)
logger = logging.getLogger("tools")

FAILURE_MODE = os.environ.get("FAILURE_MODE", "none")
API_KEY = os.environ.get("INTERNAL_API_KEY", "sk-internal-dummy-1234")   # 점
검용 더미 Secret


class RateLimitError(Exception):
    pass


def _call_backend(asset_id: str) -> dict:
    """가짜 자산 관리 백엔드 호출. FAILURE_MODE에 따라 장애를 재현한다."""
    # 주의: Secret은 절대 로그에 남기지 않습니다. 마스킹해서 기록.
    logger.info("backend 호출 asset_id=%s api_key=%s***", asset_id, API_KEY[:
6])

    if FAILURE_MODE == "timeout":
        time.sleep(5)                       # 응답 지연 재현
        raise TimeoutError("backend 응답이 5초를 초과했습니다")
    if FAILURE_MODE == "rate_limit":
        raise RateLimitError("429 Too Many Requests: 분당 호출 한도 초과")
    if FAILURE_MODE == "bad_schema":
        return {"assetStatus": "사용중인듯"}   # 계약(status, owner 키)과 다른 응답
    return {"asset_id": asset_id, "status": "사용중", "owner": "김하늘"}


@tool
def get_asset_status(asset_id: str) -> str:
```

```
    """자산 관리 백엔드에서 자산 ID로 현재 상태를 실시간 조회한다.

    Args:
        asset_id: 자산 ID (예: 'A-1001')
    """
    max_retries = 3
    for attempt in range(1, max_retries + 1):
        try:
            data = _call_backend(asset_id)

            # bad_schema 방어: 응답 계약 검증 (재시도해도 소용없으므로 즉시 fallback 안
내)
            if "status" not in data or "owner" not in data:
                logger.error("응답 스키마 불일치: keys=%s", list(data.keys()))
                return ("에러: 백엔드 응답 형식이 올바르지 않습니다. "
                        "get_asset_status_backup 도구를 대신 사용하세요.")

            return f"{data['asset_id']}: {data['status']} (사용자: {data['owne
r']})"

        except RateLimitError as e:
            wait = 2 ** attempt                     # 2초, 4초, 8초
            logger.warning("rate limit (시도 %d/%d), %d초 대기: %s", attempt, m
ax_retries, wait, e)
            if attempt == max_retries:
                return "에러: 호출 한도를 초과했습니다. get_asset_status_backup 도구
를 사용하세요."
            time.sleep(wait)

        except TimeoutError as e:
            logger.warning("timeout (시도 %d/%d): %s", attempt, max_retries,
e)
            if attempt == max_retries:
                return "에러: 백엔드 응답이 계속 지연됩니다. get_asset_status_backup
도구를 사용하세요."

        except Exception as e:
            logger.exception("예상 못한 장애")
            return f"에러: 조회 실패 ({type(e).__name__}). get_asset_status_bac
kup 도구를 사용하세요."


@tool
def get_asset_status_backup(asset_id: str) -> str:
    """백업 자산 상태 조회. get_asset_status 실패 시 사용한다. 1시간 전 스냅샷 기준이라 느
리지만 안정적이다."""
```

```
    logger.info("backup 경로 사용 asset_id=%s", asset_id)
    return f"{asset_id}: 사용중 (백업 스냅샷 기준, 갱신 주기 1시간)"
```

fallback의 핵심은 **에러 메시지 안에 다음 도구 이름을 적어 주는 것**입니다. LLM이 그 문장을 읽고 get_asset_status_backup을 스스로 호출합니다.

2. 이 도구들을 Agent에 물리는 실행 파일을 만듭니다. 아래 코드를 `day4_practice/failure_agent.py` 로 저장합니다. 정상 호출과 장애 주입을 연달아 돌리고 마지막에 DoD 통과 여부를 줄별로 출력합니다.

```
# failure_agent.py - 장애 주입 도구를 Agent에 연결해 복구 확인
import json
import os
import failure_injection
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langgraph.prebuilt import create_react_agent
from failure_injection import get_asset_status, get_asset_status_backup

load_dotenv()


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
agent = create_react_agent(
    llm,
    [get_asset_status, get_asset_status_backup],
    prompt="자산 상태 질문에는 get_asset_status를 먼저 사용하고 도구 에러 메시지에 안내된
지침을 그대로 따르세요.",
)


def print_trace(result: dict):
    """2-3에서 만든 trace 출력 (실패 지점 분석용)."""
    for i, m in enumerate(result["messages"]):
        if m.type == "human":
            print(f"{i} 사용자   {get_text(m)}")
        elif m.type == "ai" and getattr(m, "tool_calls", None):
            for tc in m.tool_calls:
                print(f"{i} 도구요청 {tc['name']} args={json.dumps(tc['args'],
ensure_ascii=False)}")
        elif m.type == "tool":
            print(f"{i} 도구결과 [{m.name}] {get_text(m)[:70]}")
        elif m.type == "ai":
            print(f"{i} 최종답변 {get_text(m)[:70]}")
```

```
def tool_names(result: dict) -> list:
    """trace에서 실제로 호출된 도구 이름만 뽑습니다."""
    return [tc["name"] for m in result["messages"]
            if getattr(m, "tool_calls", None) for tc in m.tool_calls]


def check_secret_log(path="agent.log") -> tuple:
    """로그 파일에 자격 증명 원문이 남았는지 확인합니다."""
    if not os.path.exists(path):
        return True, "agent.log 없음"
    with open(path, encoding="utf-8") as f:
        text = f.read()
    hits = [p for p in ("sk-internal-dummy-1234", "AKIA") if p in text]
    return (not hits), ("발견 0건" if not hits else f"발견: {hits}")


if __name__ == "__main__":
    question = "A-1001 자산의 현재 상태를 알려주세요."

    # 1) 정상 호출: 장애 없이 한 번 돌려 봅니다
    failure_injection.FAILURE_MODE = "none"
    normal = agent.invoke({"messages": [("user", question)]})
    normal_ok = ("get_asset_status" in tool_names(normal)
                 and "에러" not in get_text(normal["messages"][-1]))

    # 2) 실패 호출: FAILURE_MODE로 고른 장애를 주입합니다
    mode = os.environ.get("FAILURE_MODE", "timeout")
    failure_injection.FAILURE_MODE = mode
    broken = agent.invoke({"messages": [("user", question)]})
    print(f"=== 장애 주입 trace (FAILURE_MODE={mode}) ===")
    print_trace(broken)
    recovered = "get_asset_status_backup" in tool_names(broken)
    answered = bool(get_text(broken["messages"][-1]).strip())

    # 3) 로그에 Secret이 남았는지 점검합니다
    secret_ok, secret_msg = check_secret_log()

    print("\n=== DoD 판정 ===")
    print(f"정상 호출 처리     : {'통과' if normal_ok else '실패'}")
    print(f"실패 호출 처리     : {'통과' if answered else '실패'} (그래프 중단 없이 답
변 생성)")
    print(f"필수 장애 1개 복구 : {'통과' if recovered else '실패'} (장애 {mode}, 백
업 도구 호출 {'있음' if recovered else '없음'})")
    print(f"로그 Secret 점검   : {'통과' if secret_ok else '실패'} ({secret_ms
g})")
```

```
    print(f"최종                : {'DoD 충족' if all([normal_ok, answered, reco
vered, secret_ok]) else 'DoD 미충족'}")
```

3. **필수 1개**: timeout 장애를 주입하고 복구를 확인합니다.

```
$env:FAILURE_MODE="timeout"; python failure_agent.py
```

   - trace로 실패 지점을 찾아 "어느 도구가, 몇 번째 시도에서, 왜 실패했는지"를 말로 설명합니다. agent.log의 warning 기록과 trace의 ToolMessage를 함께 봅니다.
   - get_asset_status의 에러 안내 다음 턴에 get_asset_status_backup 호출이 이어지고 최종 답변이 백업 데이터로 나오는 것까지 확인합니다.
   - `failure_injection.py`의 max_retries와 대기 시간을 바꿔 로그의 시도 횟수가 따라 바뀌는 것도 확인합니다.

4. **선택 1개 이상**: 다음 중 하나 이상을 수행합니다.
   - rate_limit 복구: exponential backoff의 대기 시간이 로그에 2초, 4초, 8초로 찍히는지 확인합니다.
   - bad_schema 복구: 재시도 없이 즉시 fallback으로 넘어가는 것을 trace로 확인합니다. 기다려도 풀리지 않는 장애에 재시도는 낭비이기 때문에 분기가 다릅니다.

5. **공통 점검**: 로그에 Secret이 남지 않는지 체크리스트로 점검합니다. 점검 방법은 단순합니다. 로그 파일을 대상으로 Secret 패턴을 검색해서 한 건도 나오지 않아야 통과입니다.

```
# 1) API 키 원문이 로그에 있는지 검색 (아무것도 안 나와야 정상)
Select-String -Pattern "sk-internal-dummy-1234" agent.log

# 2) 흔한 Secret 패턴을 폭넓게 검색
Select-String -Pattern "(sk-[A-Za-z0-9]{10,}|api[_-]?key\s*[=:]\s*[A-Za-z0-
9]{8,}|password|AKIA[0-9A-Z]{16})" agent.log
```

   - [ ] 로그에 API 키 원문이 없다 (마스킹된 `sk-int***` 만 있어야 함)
   - [ ] 예외 메시지를 통째로 로그에 남길 때 예외 안에 키가 섞여 있지 않다 (HTTP 라이브러리 예외는 요청 헤더를 포함하는 경우가 있음)
   - [ ] `.env` 파일이 로그와 git 저장소에 들어가지 않는다 (`.gitignore` 확인)
   - [ ] 도구가 반환하는 문자열에도 키가 없다 (ToolMessage는 모델 API로 전송된다는 점을 기억)
   - [ ] LangSmith 같은 외부 trace 서비스를 쓴다면 거기에 올라가는 입출력에도 Secret이 없다

   만약 `logger.info(f"호출: {API_KEY}")` 같은 코드가 있었다면 1번 grep에서 바로 걸립니다. 마스킹 원칙은 앞 몇 자만 남기고 자르는 것입니다. 아예 기록하지 않으면 가장 안전합니다.

6. **마지막 5분**: 오늘 검증에 쓴 문항을 평가셋에 추가합니다. `eval_set.json`은 본인 레포(sds-ax-practice)에서 관리하는 파일이 정본입니다. 오늘 실습 폴더로 복사해 아래 질문 2개를 덧붙이고 다시 레포에 push합니다.
   레포에 이 파일이 없다면 2일차 6번째 세션의 예시 5문항으로 새로 만들어 오늘 문항을 이어 추가합니다. 이 파일은 7일차 평가 실습의 입력이 됩니다.

```
[
  {"question": "이서준 님이 소속된 팀에서 진행 중인 프로젝트를 알려주세요.", "expected_sou
rce": "search_employee,list_projects", "type": "multi_tool"},
  {"question": "A-1001 자산의 현재 상태를 알려주세요.", "expected_source": "get_ass
et_status", "type": "single_tool"}
]
```

막힐 때

- 백업 도구가 호출되지 않고 Agent가 실패했다고만 답합니다.

  get_asset_status의 에러 메시지에 백업 도구 이름이 정확히 들어 있는지 확인합니다. 시스템 프롬프트에 "도구 에러 메시지의 지침을 따르라"가 있는지도 확인합니다.

- FAILURE_MODE를 바꿨는데 동작이 그대로입니다.

  환경 변수는 실행하는 터미널 세션에만 적용됩니다. 같은 터미널에서 명령 앞에 붙여 실행했는지 확인합니다.

- `ImportError: cannot import name 'create_react_agent'`가 납니다.

  최신 langgraph에서 이름이 `create_agent`로 바뀌었습니다. `from langchain.agents import create_agent`로 바꿔 쓰되 프롬프트 인자 이름이 `prompt`가 아니라 `system_prompt` 입니다. 5일차에서 이 API를 본격적으로 씁니다.

확인: 실행 마지막의 `=== DoD 판정 ===`에서 정상 호출 처리, 실패 호출 처리, 필수 장애 1개 복구, 로그 Secret 점검 네 줄이 모두 통과로 찍히고 최종 줄이 `DoD 충족` 이면 통과입니다.
선택 항목 1개와 "어느 도구가 몇 번째 시도에서 왜 실패했는지" 설명은 trace와 agent.log를 보고 말로 확인합니다.

> 장애를 재현하고 로그와 trace로 원인을 찾아 retry와 fallback으로 복구하는 운영 감각을 익혔습니다.

▼ 모범답안 (직접 해본 뒤 열어보세요)

필수 항목인 timeout 복구를 돌렸을 때 나오는 실행 기록입니다. 여기서 읽어야 할 건 재시도 횟수와 fallback으로 넘어가는 지점입니다.

```
[시도 1] get_asset_status(asset_id='A-1008') -> TimeoutError (2초 대기 후 재시도)
[시도 2] get_asset_status(asset_id='A-1008') -> TimeoutError (4초 대기 후 재시도)
[시도 3] get_asset_status(asset_id='A-1008') -> TimeoutError
[도구 반환] 에러: 실시간 조회가 3회 모두 시간 초과했습니다. get_asset_status_backup을 대신 호출하세요.
[AIMessage] 도구 요청: get_asset_status_backup({'asset_id': 'A-1008'})
[ToolMessage] A-1008 / GPU 서버 / 사용중 (1시간 전 스냅샷)
[최종 답변] 실시간 조회가 계속 실패해 1시간 전 스냅샷 기준으로 답변드립니다. A-1008 GPU 서버는 사용중입니다.
```

"어느 도구가, 몇 번째 시도에서, 왜 실패했는지"에 대한 답안은 이 정도면 됩니다.

> get_asset_status가 3번째 시도까지 timeout으로 실패했습니다. 도구가 예외를 던지지 않고 백업 도구 이름이 들어간 문자열을 돌려줬고, 모델이 그 문장을 읽어 다음 턴에 get_asset_status_backup을 호출했습니다. 최종 답변은 1시간 전 스냅샷 기준이라는 단서를 달고 나왔습니다.

선택 항목까지 하면 세 장애의 분기 차이가 눈에 들어옵니다.

- timeout, rate_limit: 재시도를 먼저 하고 그래도 안 되면 fallback. rate_limit은 대기 시간이 2초, 4초, 8초로 늘어납니다.
- bad_schema: 재시도 없이 바로 fallback. 기다린다고 응답 형식이 바뀌지 않으므로 재시도가 낭비입니다. trace에서 시도 횟수가 1로 찍히는 걸로 확인합니다.

Secret 점검은 결과가 비어 있어야 통과입니다.

```
> Select-String -Pattern "sk-internal-dummy-1234" agent.log
(출력 없음)

> Select-String -Pattern "(sk-[A-Za-z0-9]{10,}|api[_-]?key\s*[=:]\s*[A-Za-z0-9]{8,}|password|AKIA[0-9A-Z]{16})" agent.log
(출력 없음)
```

한 건이라도 잡히면 logger 호출부에서 키를 통째로 찍는 곳을 찾아 앞 여섯 자만 남기고 자릅니다. 아예 안 남기는 쪽이 제일 안전합니다.